University of Southern Denmark

Department of Mathematics and Computer Science

SPDM801: Master Thesis in Computer Science

# Formalising $\pi$LL in Lean

*Author*

Mikkel Brix Nielsen
mikke21

*Supervisors*

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

1st June 2026

## Abstract

This thesis presents a mechanised formalisation of $\pi$LL, a process calculus relating linear logic and session-typed communication, in Lean 4. The formalisation covers the static structure of the full calculus, including session types, processes, environments, hyperenvironments, typing judgements, and substitution principles. To represent binding, the development retains explicit free identifiers while encoding bound occurrences by indices: channel binders are handled through a locally nameless representation with co-finite premises, while polymorphic type binders are treated through type-depth indexing, shifting, and substitution. The operational semantics and session-fidelity development are mechanised for the multiplicative fragment, $\pi$MLL, using a derivation-level transition system together with projected process-level and environment-level relations. The completed metatheory establishes the projection of typed transitions to process and resource transitions and develops the reverse lifting argument for all multiplicative cases except the restriction case. The remaining `res` case is isolated to the reconstruction of endpoint-bearing hyperenvironment components after a transition under restriction. The development demonstrates that mechanising the paper theory in this way can relocate implicit reasoning about bound names, freshness, and unordered linear resources into explicit structural invariants. It thereby contributes a mechanised account of $\pi$LL and identifies the remaining infrastructure required to complete session fidelity for $\pi$MLL.



## Resumé

Denne afhandling præsenterer en mekaniseret formalisering af $\pi$LL, en proceskalkule, der forbinder lineær logik med sessionstypet kommunikation, i Lean 4. Formaliseringen dækker den statiske struktur af den fulde kalkule, herunder sessionstyper, processer, miljøer, hypermiljøer, *typing judgements* og substitutionsprincipper. Til repræsentation af binding fastholder udviklingen eksplicitte frie navne, mens bundne forekomster repræsenteres ved indeks: kanalbindere håndteres gennem en *locally nameless* repræsentation med co-finite-præmisser, mens polymorfe typebindere behandles gennem typedybdeindeksering, forskydning og substitution. Den operationelle semantik og udviklingen af *session fidelity* er mekaniseret for det multiplikative fragment, $\pi$MLL, ved hjælp af et transitionssystem over *typing derivations* samt projicerede transitionsrelationer for processer og hypermiljøer. Den gennemførte metateori etablerer projektionen af typede transitioner inkulderende til proces- og ressourcetransitioner og udvikler det omvendte løfteargument for alle multiplikative tilfælde undtagen restriktionstilfældet. Det resterende `res`-tilfælde er isoleret til rekonstruktionen af de hypermiljøkomponenter, der bærer de restrikterede endepunkter, efter en transition under restriktion. Udviklingen viser, at en mekanisering af papirteorien på denne måde kan flytte implicit ræsonnement om bundne navne, friskhed og uordnede lineære ressourcer til eksplicitte strukturelle invarianter. Afhandlingen bidrager dermed med en mekaniseret redegørelse for $\pi$LL og identificerer den resterende infrastruktur, der kræves for at fuldføre sessionsfidelitet for $\pi$MLL.

# Contents

# 1 Formalisation

This section describes the mechanisation of $\pi$LL in Lean 4, bridging the theoretical foundations developed in **????** and their concrete implementation. The formalisation is organised into three top-level namespaces: `PiLL.Model`, which defines the static structure of the calculus, `PiLL.Semantics`, which defines its operational behaviour, and `PiLL.SessionFidelity`, which establishes the meta-theoretic results connecting the two. Each component is presented in dependency order, highlighting design decisions and divergences from the presentation of Montesi and Peressotti [2021].

## 1.1 Model

The `Model` namespace formalizes the static structure of the full $\pi$LL calculus and is organized around the layered architecture introduced in **??**. The mechanisation proceeds bottom-up: session types, defined in `STypes/`, provide the binding structure used by processes in `Processes/`, processes are then typed relative to environments and hyperenvironments from `Environments/` and `HyperEnvironment/`, finally, typing derivations and their metatheoretic infrastructure are collected in `Judgement/`. This includes inversion lemmas, linearity properties, substitution principles, and the co-finite freshness machinery described in **??**. Shared syntactic infrastructure used across these layers is provided by `Base.lean`. An overview of the file structure can be found in **??**.

*1.1.1 Session Types.* Session types are defined in `STypes/Basic.lean` as the inductive datatype `Types`, representing raw session-type expressions. Well-scoped use of bound type variables is characterised separately through local closure; this condition is incorporated into the mechanised typing judgement introduced in Section 1.1.5 and later shown to be preserved by typing derivations. Following the representation strategy established in **??**, type variables are isolated into a separate inductive type, `TVar`, which explicitly distinguishes between free and bound occurrences:

```
inductive TVar : Type
  | free  (x : FTVar)
  | bound (x : BTVar)
```

Here, `FTVar` represents a free type variable identified by a natural number, while `BTVar` represents a bound type variable encoded as a de Bruijn index.

The remaining constructors directly mirror the theoretical grammar of the calculus. In addition, the formalisation introduces uninterpreted atomic types (`atom` and `atomDual`), representing protocol leaves not further decomposed by the session type grammar and providing the base cases for the inductive structure of the syntax. Table 1 illustrates a representative correspondence between the paper syntax, the locally nameless representation, and the Lean implementation.

**Table 1.** Comparison of standard, locally nameless, and Lean representations for $\pi$LL types.

| Standard | Locally Nameless | Lean Constructor | Binds |
|---|---|---|---|
| $A \otimes B$ | $A \otimes B$ | `\| tensor (A B : Types)` | None |
| $1$ | $1$ | `\| one` | None |
| $\forall X.A$ | $\forall.A$ | `\| forall_ (A : Types)` | 1 type |
| $\exists X.A$ | $\exists.A$ | `\| exists_ (A : Types)` | 1 type |
| $X$ | $X$ | `\| var (v : TVar)` | None |

Notably, the polymorphic binders $\forall.A$ and $\exists.A$ reflect the depth-indexed approach detailed in **??**. As shown in **??**, the corresponding Lean constructors keep the bound de Bruijn index implicit, taking only the continuation body `A` as an argument. An underscore "_" is appended to the constructor names to prevent namespace collisions with Lean's built-in `forall` and `exists` keywords.

*Duality.* `Types.dual` implements duality as a structurally recursive function, mapping each constructor to its dual. For example:

```
def Types.dual : Types → Types
  | .atom a     => .atomDual a
  | .atomDual a => .atom a
  | .tensor A B  => .parr    (dual A) (dual B)
  | .forall_ A   => .exists_ (dual A)
  | .one         => .bot
  | ...
```

The fundamental property that duality is an involution, $(A^{\perp})^{\perp} = A$, is proved in `STypes/Lemmas.lean` by structural induction.

The formalisation additionally establishes that duality preserves local closure and commutes with both shifting and substitution:

$$A.\mathtt{lc} \iff A^{\perp}.\mathtt{lc} \qquad \uparrow_d^c (A^{\perp}) = (\uparrow_d^c A)^{\perp} \qquad B^{\perp}\{A/k\} = (B\{A/k\})^{\perp}$$

*Local closure.* `STypes/Properties.lean` defines local closure for types as a depth-indexed predicate `Types.lc (k : Nat) : Types → Prop`, where the depth parameter $k$ tracks how many binders the recursion has traversed.

At the leaves of the syntax tree, local closure is evaluated by the helper predicate `TVar.lc`. Because free variables are explicit they do not contain any indices, as such they are inherently locally closed at any depth. By contrast, a bound variable is only locally closed if its de Bruijn index $i$ is strictly less than the current depth $k$, ensuring it points to an enclosing scope:

```
def TVar.lc : Nat → TVar → Prop
  | _, .free _   => True
  | k, .bound i  => i < k
```

The primary `Types.lc` predicate is then defined by structural recursion. For non-binding constructors, the depth parameter distributes across the sub-terms. When a variable is encountered, the predicate delegates the check to `TVar.lc`. When descending into the body of a polymorphic binder, the depth counter increments by one, bringing the newly introduced index safely into scope:

```
def Types.lc : Nat → Types → Prop
  | _, .one         => True
  | k, .var v       => TVar.lc k v
  | k, .tensor A B  => A.lc k ∧ B.lc k
  | k, .forall_ A   => A.lc (k + 1)
  | k, .exists_ A   => A.lc (k + 1)
  | ...
```

Finally, a type is defined as *closed* if it contains no dangling indices at the top level, meaning it is locally closed at depth 0.

*Shifting.* `STypes/Shift.lean` defines shifting following the standard de Bruijn shifting operation introduced in **??**. The operation traverses the syntax tree structurally using a cutoff depth $d$ and a shift amount $c$.

For standard connectives, shifting distributes recursively across subterms, while base constructors such as `one` and `bot` remain unchanged. When traversing a polymorphic binder like `forall_`, the cutoff depth increments by one to account for the newly entered scope:

```
def Types.shift (d c : Nat) : Types → Types
  | .var v       => .var (v.shift d c)
  | .tensor A B  => .tensor (A.shift d c) (B.shift d c)
  | .forall_ A   => .forall_ (A.shift (d + 1) c)
  | t            => t
  | ...
```

At the variable level, free variables remain unchanged. For bound indices, the function compares the de Bruijn index $i$ against the cutoff depth $d$. Indices satisfying $i < d$ are locally bound within the traversed scope and therefore remain unchanged. Otherwise, the index is incremented by $c$ to preserve its reference to the surrounding environment:

```
def TVar.shift (d c : Nat) : TVar → TVar
  | .bound i => if i < d then .bound i else .bound (i + c)
  | .free n  => .free n
```

The operation is exposed through the common `HasShift` interface, allowing the notation $A \uparrow d, c$, $A \uparrow c$ and $A^+$ to be used uniformly for types and, later, for processes, environments, and hyperenvironments. The shared notation declarations are given in **??**.

*Substitution.* `STypes/Subst.lean` defines substitution by replacing a target de Bruijn index $k$ in a host type $B$ with a type $A$, denoted $B\{A//k\}$. At the variable level, the function compares a bound de Bruijn index $i$ with the target depth $k$. If $i = k$, the target index has been reached and the variable is replaced by $A$. If $i < k$, the variable is bound by an inner scope and is left unchanged. Finally, if $i > k$, the index refers to an outer binder beyond the substituted variable. Since substitution removes the binder corresponding to $k$, such indices are decremented by one to preserve their relative distance to the surrounding environment.

```
def Types.subst (B A : Types) (k : Nat) : Types :=
  match B with
  | .var (.free n)  => .var (.free n)
  | .var (.bound i) =>
      if i == k then A
      else if i > k then .var (.bound (i - 1))
      else .var (.bound i)
  | .tensor A B => .tensor (A.subst A k) (B.subst A k)
```

The operation is lifted to the full `Types` syntax by structural recursion. For ordinary connectives, substitution distributes across subterms. When descending into a polymorphic binder such as `forall_`, the target depth is incremented to $k + 1$, and the substituting type $A$ is shifted by one. This prevents the free indices in $A$ from being captured by the newly entered binder:

```
  | .forall_ B => .forall_ (B.subst (shift 0 1 A) (k + 1))
```

The interaction between substitution and local closure is captured by `Types.lc_subst_inv`. Given a type $A$ that is locally closed at depth $n + k$, the lemma relates local closure before and after

substituting $A$ for index $k$ in $B$: $(B\{A/k\}).\texttt{lc}\ (n+k) \iff B.\texttt{lc}\ (n+k+1)$. Intuitively, substituting for index $k$ essentially peals of a type binder, as such local closure at depth $n+k$ after substitution corresponds to local closure at depth $n+k+1$ before substitution. A specialisation of this result is used later to show that typing judgements contain locally closed syntax.

As with shifting, substitution is exposed through the common `HasSubst` interface, allowing the notation $B\{A//k\}$ to be reused when substitution is lifted to processes, environments, and hyperenvironments (see **??**).

*Notation.* `STypes/Notation.lean` introduces notation mirroring the mathematical presentation from **??**. The only divergence is that exponentials are written $!!A$ and $??A$ to avoid clashes with Lean syntax. The corresponding Lean notation declarations are given in **??**.

*1.1.2 Processes.* Processes are defined in `Processes/Basic.lean` as the inductive datatype `Proc`, representing raw process terms independently of whether they are well typed. Following the established locally nameless architecture, channel references are partitioned into a dedicated `Channel` inductive. Explicit free channels are instantiated as concrete natural numbers (`FPName`). While the raw process syntax permits arbitrary reuse of these numbers, using `Nat` allows the formalisation to systematically compute strictly fresh names by incrementing the supremum of any finite set of existing names, as mechanised in `Processes/Fresh.lean`. Conversely, bound channels are represented using de Bruijn indices, encoding their relative distance to the corresponding binder.

```
inductive Channel : Type where
  | free  (x : FPName)
  | bound (x : BPName)
```

As discussed in **??**, binding constructs in the process syntax no longer carry explicit bound channel names. Instead, bound occurrences are captured purely by their relative position. Comparing the standard paper grammar with the intermediate locally nameless notation and the final constructors of the `inductive Proc` type illustrates this abstraction:

| **Standard** | **Locally Nameless** | **Lean Constructor** | **Binds** |
|---|---|---|---|
| $x[y].P$ | $x[\bullet].P$ | `\| tensor (x : Channel) (P : Proc)` | 1 channel |
| $x(y).P$ | $x(\bullet).P$ | `\| parr (x : Channel) (P : Proc)` | 1 channel |
| $\nu xy\,P$ | $\nu(\bullet,\bullet)\,P$ | `\| cut (P : Proc)` | 2 channels |
| $x(X).P$ | $x(\diamond).P$ | `\| input (x : Channel) (P : Proc)` | 1 type |
| $x[\text{DUP}](x').P$ | $x[\text{DUP}](\bullet).P$ | `\| duplicate (x : Channel) (P : Proc)` | 1 channel |

Notice that `cut` takes only a continuation $P$ with no explicit channel arguments, as both dual endpoints are simultaneously bound as indices 0 and 1 within the scope of $P$. Similarly, for constructs like `tensor`, `parr`, and `input`, the subject channel $x$ over which the interaction occurs is kept explicit, while the name or type variable bound in the continuation is anonymized and represented by an index. The full inductive definition is listed in **??**.

*Opening and closing.* `Processes/OpenClose.lean` defines opening and closing operations following the locally nameless principles established in **??**. In the theoretical presentation, opening a process at depth $k$ with $x$ is denoted as $\{k \mapsto x\}P$, and conversely, abstracting a specific name back into a bound index is denoted as $\{k \leftarrow x\}P$.

To mechanise this in Lean, we denote explicit free channels as $\#x$ (mapping to `Channel.free x`) and introduce custom notation for these substitution operations. Expressions such as $P(\!(k|\#x)\!)$

and $P(\!(k|\#x, \#y)\!)$ denote the opening of a process by instantiating the $k$-th (and $k+1$-th) bound indices with free channels. Conversely, $P\langle\!\langle k|\#x\rangle\!\rangle$ and $P\langle\!\langle k|\#x, \#y\rangle\!\rangle$ denote the closing of a process by abstracting $x$ and $y$ back into de Bruijn indices. For convenience, the depth parameter can be omitted when operating at the innermost binder ($k = 0$), simplifying the notation to $P(\!(\#x, \#y)\!)$ and $P\langle\!\langle \#x, \#y\rangle\!\rangle$. The typeclasses and notation declarations supporting these overloaded operations are collected in **??**.

Because binders are represented positionally, the depth parameter $k$ increments according to the number of channels bound by each constructor. For standard communication prefixes like `tensor`, `parr`, and the server `duplicate` operation, exactly one channel is bound, incrementing $k$ by one. The restriction constructor (`cut`) simultaneously binds two dual endpoints, requiring $k$ to increment by two. The `input` constructor binds a type variable rather than a channel and therefore leaves $k$ unchanged. The `server` constructor additionally carries an explicit set of dependency channels `zs : Finset Channel`, representing the replicable processes the server depends on, which must be acted on simultaneously with $x$ and $P$.

```
def Proc.open (P : Proc) (u : Channel) (k : Nat) : Proc :=
  match P with
  | .tensor x P     => .tensor (x.open u k) (P.open u (k + 1))
  | .parr   x P     => .parr   (x.open u k) (P.open u (k + 1))
  | .cut      P     => .cut                 (P.open u (k + 2))
  | .duplicate x P  => .duplicate (x.open u k) (P.open u (k + 1))
  | .server x zs P  => .server (x.open u k) (zs.open u k) (P.open u k)
  | .input  x P     => .input  (x.open u k) (P.open u k)
  | ...
```

For channel occurrences, the recursive traversal delegates to `Channel.open`, for the dependency set carried by a server, it uses the pointwise lifting `Finset.open`. For bound channels, the de Bruijn index $i$ is compared against the target depth $k$ and replaced with $v$ upon an exact match. For dependency sets, `Channel.open` is mapped pointwise. In both cases free channels remain entirely unaffected:

```
def Channel.open (u v : Channel) (k : Nat) : Channel :=
  match u with
  | bound i => if i == k then v else bound i
  | c => c

def Finset.open (zs : Finset Channel) (v : Channel) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.open v k)
```

For restriction, the two-name opening interface is instantiated by sequential opening, replacing index $k+1$ with the second endpoint $v$, then replacing index $k$ with the first endpoint $u$:

```
instance : HasOpenTwo Proc Channel Channel Nat where
  open_ P u v k := (Proc.open (Proc.open P v (k + 1)) u k)
```

To complete the bidirectional infrastructure, `Proc.close`, `Channel.close`, and `Finset.close` define the corresponding closing operation. `Proc.close` traverses the process tree using a structural recursion identical to `Proc.open`, applying the same depth-incrementing rules. At the leaves, `Channel.close` compares each free channel against the target name $x$ and abstracts it into a bound de Bruijn index at depth $k$ upon a match, while bound indices are left unchanged. Dependency sets are handled pointwise by `Finset.close`:

```
def Channel.close (u : Channel) (x : FPName) (k : Nat) : Channel :=
  match u with
  | .free name => if x == name then .bound k else .free name
  | .bound i   => .bound i

def Finset.close (zs : Finset Channel) (x : FPName) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.close x k)
```

*Local closure.* `Processes/LocalClosure.lean` defines local closure for processes as a two-parameter predicate `Proc.lc (k n : Nat) : Proc → Prop`. Because the calculus features a two-sorted syntax, the predicate must maintain independent depth counters: $k$ tracks the depth of channel binders, while $n$ tracks the depth of type binders. The latter is required because processes may contain embedded types whose free indices must be interpreted relative to the surrounding type-binder depth.

The structural recursion evaluates each constructor by incrementing the appropriate depth counter according to the number and sort of binders introduced. Channel binders increase the channel depth $k$, while type binders increase the type depth $n$. For example, `tensor` binds a single continuation channel, incrementing the depth to $k + 1$, whereas `cut` binds two dual endpoints simultaneously, incrementing the depth to $k + 2$. Conversely, `input` binds a type variable, leaving $k$ unchanged while incrementing $n$ to $n + 1$. The `server` constructor therefore requires local closure of `x`, `zs`, and `P`. Constructors introducing no binders leave both counters unchanged.

```
def Proc.lc : Nat → Nat → Proc → Prop
  | _, _, .nil            => True
  | k, n, .one x P        => x.lc k ∧ P.lc k n
  | k, n, .tensor x P     => x.lc k ∧ P.lc (k + 1) n
  | k, n, .cut P          => P.lc (k + 2) n
  | k, n, .input x P      => x.lc k ∧ P.lc k (n + 1)
  | k, n, .server x zs P  => x.lc k ∧ zs.lc k ∧ P.lc k n
  | ...
```

At the leaves, `Channel.lc` checks that any bound index is strictly within the current depth, while free channels are trivially locally closed. For dependency sets, `Finset.lc` requires every channel in the set to satisfy `Channel.lc k`:

```
def Channel.lc : Nat → Channel → Prop
  | _, .free _   => True
  | k, .bound i  => i < k

def Finset.lc (zs : Finset Channel) (k : Nat) : Prop :=
  ∀ z ∈ zs, z.lc k
```

A process is considered globally *closed* when it contains no dangling indices of either sort, meaning `Proc.lc 0 0` holds.

*Free names and freshness.* `Processes/Names.lean` defines `Proc.f : Proc → Finset FPName` to compute the free channel names of a process. The function traverses the syntax tree structurally, collecting all explicit free channel identifiers into a finite set. Since locally nameless binders are represented using anonymous indices, bound channels contribute nothing to this set.

For constructors carrying explicit subject channels, such as `tensor` and `link`, the free names of those channels are combined with the recursively computed free names of their continuations.

Constructors without explicit channel identifiers of their own, such as `cut`, contribute only the free names recursively obtained from their continuations. The `server` constructor contributes the free names of its subject channel, its dependency set and continuation:

```
def Proc.f : Proc → Finset FPName
  | .cut P          => P.f
  | .par P Q        => P.f ∪ Q.f
  | .link x y       => x.f ∪ y.f
  | .tensor x P     => x.f ∪ P.f
  | .one x P        => x.f ∪ P.f
  | .server x zs P  => x.f ∪ zs.f ∪ P.f
  | .nil        => {}
  | ...
```

At the leaves, `Channel.f` returns the singleton set for free channels and the empty set for bound indices. For dependency sets, `Finset.f` collects free names across all channels in the set:

```
def Channel.f : Channel → Finset FPName
  | .free x  => {x}
  | .bound _ => {}

def Finset.f (zs : Finset Channel) : Finset FPName :=
  zs.biUnion Channel.f
```

The resulting finite set is used by `Processes/Fresh.lean` to generate names outside a forbidden context. Since free process names are natural numbers, a fresh name for a finite set $L$ is computed as `freshName L` $= \sup(L) + 1$, with `fresh_is_fresh` confirming that every element of $L$ is strictly smaller than this value. The lemmas `exists_one_fresh` and `exists_two_fresh` then guarantee one or two distinct names outside $L$, which becomes relevant later when having to instantiate co-finite quantification premises, in particular for terms like restriction, where two fresh and distinct endpoints must be introduced simultaneously.

*Substitution.* The process syntax supports two substitution operations, corresponding to the occurrence of both channel names and session types in process terms. Name substitution is defined in `Processes/SubstNames.lean` as the function `Proc.substNames (R T : FPName) : Proc → Proc`, replacing all explicit free occurrences of the target name $T$ with the replacement name $R$. Since bound channels are represented by de Bruijn indices, this operation acts only on explicit free channel leaves and does not require $\alpha$-renaming in order to pass under channel binders, since a substituted free name cannot be captured by a binder represented as an index. Name substitution alone does not, however, ensure that the replacement name is fresh for the remaining free names or compatible with the linear resource structure of a typing judgement. Such conditions must be established separately when substitution is used in typing or metatheoretic arguments.

At the leaves, `Channel.subst` replaces a matching free channel and leaves bound indices unchanged, while dependency sets are handled pointwise by `Finset.subst`:

```
def Channel.subst (R T : FPName) : Channel → Channel
  | .free x  => if x == T then .free R else .free x
  | .bound i => .bound i

def Finset.subst (zs : Finset Channel) (R T : FPName) : Finset Channel :=
  zs.image (fun u => u.subst R T)
```

Type substitution is defined in `Processes/SubstTypes.lean` as `Proc.substTypes (A : Types) (k : Nat) : Proc → Proc`, substituting $A$ for the de Bruijn type index $k$ throughout a process. Whenever the traversal encounters an embedded type, such as the type argument of an `output` prefix, it delegates to the type-level substitution operation `Types.subst`. When crossing an `input` prefix, which binds a type variable, the target depth is incremented and the substituted type is shifted, mirroring the binder case of `Types.subst` from Section 1.1.1:

```
def Proc.substTypes (A : Types) (k : Nat) : Proc → Proc
  | .input x P => .input x (P.substTypes (A.shift 0 1) (k + 1))
```

A consequence of this representation is that no analogous channel shifting operation is required. In a pure de Bruijn encoding, variables are represented as relative indices and must be shifted when crossing binders. Free channels, however, are represented as explicit identifiers rather than relative positions, so their representation is unaffected by binder depth. This avoids structural adjustment of channel names during substitution and simplifies the process-level infrastructure.

*Type shifting.* `Processes/SubstTypes.lean` also defines `Proc.shiftTypes (d c : Nat) : Proc → Proc`, which lifts the type-level shifting operation to processes. The traversal is uniform across most constructors, delegating to `Types.shift` whenever an embedded type is encountered, such as the type argument of an `output` prefix. The only non-trivial case is `input`, which binds a type variable and therefore increments the cutoff depth $d$ before traversing its continuation, mirroring the binder case of `Types.shift`:

```
def Proc.shiftTypes (d c : Nat) : Proc → Proc
  | .output x P A  => .output x (P.shiftTypes d c) (A.shift d c)
  | .input x P     => .input x (P.shiftTypes (d + 1) c)
  | ...
```

*Structural Properties.* `Processes/Lemmas.lean` establishes the fundamental structural properties required by the later typing and operational metatheory. The lemmas fall into three groups.

The first group supports the typing invariants. The lemmas `Proc.f_open_erase` and `Proc.f_open_two_erase` establish that opening a process with fresh names and then erasing those names from its free-name set recovers the original free names. They are used in the proof of `Typing_f_eq_names`. The lemmas `Proc.lc_of_open_gen` and `Proc.lc_of_open_two` show that opening a locally closed abstraction with locally closed names yields a locally closed process. They therefore support the proof of `Typing_preserves_lc_proc`, which establishes that well-typed processes are locally closed.

The second group supports the substitution lemmas. The lemmas `Proc.open_substNames_comm` and `Proc.openCut_substNames_comm` establish that opening commutes with name substitution. Their counterparts for type substitution, `Proc.open_substTypes_comm` and `Proc.openCut_substTypes_comm`, establish the analogous property for substitution of types under binders. These results are used in the typability proof, where co-finite witnesses from the typing rules must be reconciled with the concrete names introduced by process transitions.

The final group concerns freshness under opening. The lemma `Proc.not_mem_f_open` shows that opening introduces only the selected name. Thus, a different name that is absent from the original process remains absent from the opened continuation. This fact is used later when transitions and typing proofs reason about opened binder bodies under freshness side conditions.

*Notation.* `Processes/Notation.lean` introduces process prefix notation mirroring the locally nameless syntax established in **??**. For example, $\#x[\![\bullet]\!].P$ maps to `Proc.tensor (#x) P` and

$\nu(\!(\bullet,\bullet)\!)\ P$ maps to `Proc.cut P`. Parallel composition and the terminated process are written $P \mid_p Q$ and $\mathbf{0}$ respectively. The full notation is listed in **??**.

*1.1.3 Environments.* Environments are defined in `Environment/Basic.lean` as `Env`, an abbreviation for a list of channel-type pairs:

```
abbrev Elem := (FPName × Types)
abbrev Env  := List Elem
```

The choice of `List` as the underlying representation deserves explanation. Theoretical environments are unordered and duplicate-free, suggesting `Finset` as a natural implementation. However, since `Finset` is implemented as a quotient over lists, it complicates structural induction, pattern matching, and inversion over environments. This proved inconvenient in a development where typing derivations are frequently inverted and typing environments must be rearranged explicitly. A second attempt using association lists (`AList`) encountered similar proof-engineering difficulties. The final design therefore uses plain lists and recovers the unordered structure through an explicit permutation relation, trading quotient reasoning for structural induction and simpler interaction with Lean's tactic machinery.

*Recovering the PCM structure.* Environment composition is implemented as list concatenation, yielding a simple total merge operation:

```
abbrev Env.merge (Γ Δ : Env) : Env := Γ ++ Δ
infixl:69 "," => Env.merge
```

In the formalisation, environment composition is represented by list concatenation. Because Lean reserves the ordinary comma for syntactic purposes, the concrete notation uses the comma-like symbol "`,`", as in $[\Gamma\,,\Delta]$. However, the ordinary comma is what is used to render this operation throughout the thesis.

This representation does not make environments a partial commutative monoid by definition. List concatenation is total and order-sensitive, meaning that associativity and identity are inherited from concatenation, while commutativity must be recovered up to permutation. The formalisation therefore uses Lean's built-in `List.Perm` relation, exposed through the `HasPerm` interface and written $\Gamma \sim \Delta$. The corresponding monoidal laws include:

```
lemma Env.merge_unitL (Γ : Env) : ∅,Γ = Γ
lemma Env.merge_unitR (Γ : Env) : Γ,∅ = Γ
lemma Env.merge_assoc (Γ Δ Ξ : Env) : Γ,Δ,Ξ = Γ,(Δ,Ξ)
lemma Env.merge_comm (Γ Δ : Env) : Γ,Δ ~ Δ,Γ
```

The partiality condition is enforced separately by predicates on the merged resources. The function `Env.names` extracts the domain of an environment as a finite set of channel names. Mutual compatibility is expressed by disjointness of these domains, while internal linearity is expressed by the absence of duplicate names:

```
def Env.names (Γ : Env) : Finset FPName := (Γ.map Prod.fst).toFinset
def Env.disjoint (Γ Δ : Env) : Prop := Disjoint Γ.names Δ.names
def Env.Nodup (Γ : Env) : Prop := (Γ.map Prod.fst).Nodup
```

The connection between these predicates and merge is captured by `Env.Nodup_merge_iff`, which states that a merged environment is linear exactly when both component environments are individually linear and their domains are disjoint:

```
lemma Env.Nodup_merge_iff {Γ Δ : Env} :
  (Γ,Δ).Nodup ↔ (Γ.Nodup ∧ Δ.Nodup ∧ Γ.disjoint Δ) := by ...
```

The proof reduces directly to the corresponding property of list append, `List.nodup_append`, together with the finite-set formulation of environment-domain disjointness.

*Server usability.* The typing rule for server replication requires all dependency channels to carry exponential request types. This side condition is captured by `Env.serverUsable`, which asserts that every type in an environment satisfies `Types.isServerUsable`, i.e. has the form $?B$.

```
def Env.serverUsable (Γ : Env) : Prop :=
  ∀ p, p ∈ Γ → p.snd.isServerUsable
prefix:max "?ₑ" => Env.serverUsable
```

*Lifted operations.* Since environments contain types, the locally nameless operations from Section 1.1.1 are lifted pointwise over the type component of each `Elem`. Local closure, type shifting, name substitution, and type substitution are all defined by mapping over the list. The file `Environment/Lemmas.lean` proves that these operations preserve the names, disjointness, nodup, and permutation properties of environments.

The interaction between shifting and local closure for environments is captured by `Env.lc_shift_inv`. When an environment is moved under a newly introduced type binder, shifting adjusts its bound type indices so that its scoping structure is preserved at the increased binder depth. Formally, the lemma states that $(\Gamma^{+}).\mathtt{lc}\ (n+k+1) \iff \Gamma.\mathtt{lc}\ (n+k)$. Thus, local closure of the shifted environment at the increased depth corresponds precisely to local closure of the original environment at the preceding depth. A specialisation of this result is used later to show that typing judgements contain only locally closed syntax.

### 1.1.4 *Hyperenvironments.*

Hyperenvironments are defined in `HyperEnvironment/Basic.lean` as `HyperEnv`, an abbreviation for a list of environments:

```
abbrev HyperEnv := List Env
abbrev HyperEnv.merge (𝒢 ℋ : HyperEnv) : HyperEnv := 𝒢 ++ ℋ
infixl:55 " |ₕ " => HyperEnv.merge
```

As for environments, merging is implemented as list concatenation, using $\varnothing$ as the unit and $\mathcal{G}|_h\mathcal{H}$ as notation for composition. Associativity and unit laws therefore follow from the underlying list structure. However, recovering the commutative structure requires more than the ordinary list permutation relation, since hyperenvironments are lists whose elements are themselves environments considered modulo permutation.

*Deep permutation.* For environments, Lean's `List.Perm` relation is sufficient, since it allows arbitrary reordering of channel-type pairs. For hyperenvironments, however, permutation must operate at two levels: it must allow reordering of component environments, and it must also respect permutation equivalence inside each component environment.

For example, consider the hyperenvironment containing a single component environment $[x:A, y:B]$. Ordinary `List.Perm` treats this component environment as an opaque element, so it cannot identify it with $[y:B, x:A]$, which would be the desired behaviour. Thus, a custom relation `HyperEnv.Perm`, which permits component environments to be replaced by permutation-equivalent environments, was developed.

```
inductive HyperEnv.Perm : HyperEnv → HyperEnv → Prop where
  | nil  : Perm [] []
```

```
  | cons : {Γ Δ : Env} {𝒢 ℋ : HyperEnv} : Γ ~ Δ → Perm 𝒢 ℋ → Perm (Γ :: 𝒢) (Δ :: ℋ)
  | swap : {Γ Δ : Env} {𝒢 : HyperEnv} : Perm (Γ :: Δ :: 𝒢) (Δ :: Γ :: 𝒢)
  | trans {𝒢 ℋ 𝒥 : HyperEnv} : Perm 𝒢 ℋ → Perm ℋ 𝒥 → Perm 𝒢 𝒥
```

The key constructor is cons. Unlike ordinary list permutation, it does not require the head environments to be syntactically identical, it only requires them to be permutation-equivalent. Thus `HyperEnv.Perm` is a deep permutation relation, propagating equivalence both across the ordering of component environments and within the environments themselves.

*Well-formedness.* Hyperenvironment well-formedness is decomposed into two predicates. The predicate `HyperEnv.Nodup` requires each component environment to be internally linear, while `HyperEnv.PairwiseDisjoint` requires distinct component environments to have disjoint domains:

```
def HyperEnv.Nodup (𝒢 : HyperEnv) : Prop :=
  ∀ Γ, Γ ∈ 𝒢 → Env.Nodup Γ

def HyperEnv.PairwiseDisjoint (𝒢 : HyperEnv) : Prop :=
  List.Pairwise Env.disjoint 𝒢
```

The distinction is important because the two predicates rule out different violations of linearity. For example, assuming $x$, $y$, and $z$ are distinct names:

- $[x:A] \mid_h [y:B, z:C]$ satisfies both predicates.
- $[x:A, x:B] \mid_h [y:C]$ violates Nodup ($x$ appears twice within one component).
- $[x:A] \mid_h [x:B, y:C]$ violates `PairwiseDisjoint` ($x$ appears within distinct components).

Together, these predicates enforce global linearity in that every channel it appears at most once across the whole hyperenvironment.

*Lifted operations.* Type shifting, name substitution, and type substitution are lifted pointwise from environments to hyperenvironments. The accompanying lemmas in `HyperEnvironment/Lemmas/Basic.lean` establish how these operations interact with the structural properties required later, including names, disjointness, pairwise disjointness, and deep permutation. Since the definitions follow uniformly from their environment-level counterparts, they are omitted here and included in the accompanying Lean archive.

### 1.1.5 *Typing Judgements.*

The typing relation is defined in `Judgement/Basic.lean` as the inductive predicate `Typing`:

```
inductive Typing : Nat → Proc → HyperEnv → Prop
```

We write $n \vdash P :: \mathcal{G}$ for the judgement that $P$ is typed under the hyperenvironment $\mathcal{G}$ at type-binder depth $n$. An inhabitant of this proposition is a typing derivation. The index $n$ tracks the current type-binder depth, while the remaining indices record the process and hyperenvironment appearing in the judgement. These indices are later exposed by the projection functions used to relate typed transitions to the process-level and environment-level semantics. The full inductive definition, including the typing constructors and exchange rules, is given in **??**.

The process syntax and typing derivations occupy distinct levels of the formalisation. Terms of type `Proc` represent raw process syntax and may be constructed independently of whether they admit a typing derivation. By contrast, a term inhabiting $n \vdash P :: \mathcal{G}$ is a derivation that $P$ uses the resources in $\mathcal{G}$ according to the typing rules: each constructor requires the typed premises and structural side conditions needed for its conclusion. Properties such as local closure, linearity, and correspondence between free process names and hyperenvironment names are therefore established as invariants of valid typing derivations rather than imposed on the raw syntax datatype.

*Type-binder depth.* The type-binder depth is relevant whenever the derivation crosses a polymorphic binder. For example, the `forall_` constructor increments the depth from $n$ to $n + 1$ and shifts the surrounding environment to keep embedded type indices in scope:

```
| forall_ :
    Typing (n + 1) P [x:B :: Γ⁺] →
    Typing n (#x(( ⋄ )).P) [x:∀.B :: Γ]
```

The shift $\Gamma^+$ abbreviates shifting the type indices in $\Gamma$ by one starting at depth 0, mirroring the binder-sensitive shifting used by type substitution in Section 1.1.1.

The type-binder depth is also relevant for the **??** rule, which requires both sub-derivations to share the same depth $n$. When derivations are available at different depths, they can be brought into alignment using `Typing_weakening`. This lemma states that a derivation at depth $n$ can be lifted to depth $n + c$ by shifting the embedded type indices in both the process and the hyperenvironment:

```
lemma Typing_weakening :
  Typing n P 𝒢 → ∀ d c, Typing (n + c) (P ↑ d, c) (𝒢 ↑ d, c) := by ...
```

This provides the weakening principle needed to compose derivations across polymorphic binders while keeping embedded type indices in scope.

*Exchange and explicit side conditions.* In some constructors, the formalised rule chooses an ordering different from the paper presentation so that the entry manipulated by the rule occurs at the head of the list representing its environment component. This is an implementation-oriented choice and does not affect the typing discipline. The order-insensitive interpretation of environments and hyperenvironments is recovered by exchange rules in the typing relation, which permit reasoning up to permutation:

```
| exchange_env   : Typing n P (Γ :: 𝒢) → Γ ~ Δ → Typing n P (Δ :: 𝒢)
| exchange_hyper : Typing n P 𝒢 → 𝒢 ~ ℋ → Typing n P ℋ
```

The exchange rules recover the unordered character of the paper-based presentation without quotienting environments or hyperenvironments. The complementary issue is partiality. This stems from merging being implemented as total list concatenation. Thus, typing rules have been formalised to carry explicit disjointness and freshness premises to ensure that only valid linear compositions are formed. For example, `mix` requires the two hyperenvironments to be disjoint, while prefix rules such as `tensor` and `parr` require freshness premises ensuring that subject channels do not already appear in the continuation environment.

The same design principle reappears later in the operational semantics, where transitions must also respect permutation equivalence of environments and hyperenvironments.

*Co-finite quantification.* Binding rules use co-finite quantification as described in **??**. For example, the `parr` constructor types an `input` prefix by requiring the opened continuation to be well typed for every fresh channel name outside a finite set $L$:

```
| parr (hF : x ∉ Γ.names) (L : Finset FPName) :
    (∀ y ∉ L, Typing n (P(( #y ))) [y:A :: x:B :: Γ]) →
    Typing n (x(( • )).P) [x:A ⅋ B :: Γ]
```

Here, the process carries no explicit bound channel name. Instead, the bound occurrence in the continuation is instantiated by opening $P$ with a fresh name $y$. The premise `hF` enforces that the subject channel $x$ is not already present in the continuation environment. Rules binding multiple names, such as `cut`, follow the same pattern but quantify over two distinct fresh names.

*Structural invariants.* `Judgement/Names.lean` establishes the central name invariant for the typing relation, named `Typing_f_eq_names`. It states that, for any well-typed process, the free names of the process coincide exactly with the domain of its hyperenvironment.

$$n \vdash P :: \mathcal{G} \implies P.f = \mathcal{G}.names$$

The proof proceeds by induction on the typing derivation. Most cases follow by unfolding `Proc.f` and `HyperEnv.names` and rewriting with the induction hypothesis. In the co-finite binding cases, the proof instantiates fresh names using `exists_two_fresh`, applies the induction hypothesis to the opened process, and then uses `Proc.f_open_erase` to cancel the introduced names from the free name set and recover equality for the closed process.

This ensures exact correspondence between process free names and typed environment entries: no free process channel is left untyped, and no environment entry is unused.

Building on this, `Judgement/Properties/Linearity.lean` proves that any well-typed hyperenvironment is pairwise disjoint and all component environments have no duplicate names.

$$n \vdash P :: \mathcal{G} \implies \mathsf{Nodup}\ \mathcal{G} \wedge \mathcal{G}.\mathsf{PairwiseDisjoint}$$

The proof proceeds by induction on the typing derivation. Most non-binding rules are discharged uniformly by unfolding the relevant definitions and applying the induction hypothesis. The co-finite binding cases (`cut`, `tensor`, `parr`, and contraction (`c`)) require instantiating fresh names and reconstructing both nodupness and disjointness of the component environments from the opened induction hypothesis. The exchange cases use preservation of these properties under permutation.

Together, `Typing_f_eq_names` and `Typing_preserves_linearity` guarantee that typing derivations can only be constructed for well-formed, linear contexts. In particular, a well-typed process cannot contain duplicate free channel occurrences: by the name correspondence, duplicate free names would require duplicate entries in the hyperenvironment, contradicting the linearity constraints.

A third invariant, `Typing_preserves_lc`, establishes that every well-typed process is locally closed and that the types occurring in each component of its hyperenvironment are locally closed at the current type-binder depth. The proof is split into `Typing_preserves_lc_proc` for the process and `Typing_preserves_lc_context` for the hyperenvironment, and the two results are then packaged together. Both proofs proceed by induction on the typing derivation. In the co-finite binding cases, the typing premise provides local closure only for continuations opened with sufficiently fresh names. The lemmas `Proc.lc_of_open_gen` and `Proc.lc_of_open_two` recover local closure of the corresponding binder-containing process. The polymorphic cases additionally require inversion lemmas for the transformations appearing in their premises. The lemma `Env.lc_shift_inv_0`, derived pointwise from `Types.lc_shift_inv_0`, recovers local closure of an environment before it is shifted beneath a type binder. Given local closure of the substituted type, `Types.lc_subst_inv_0` recovers local closure of a type body at depth $n + 1$ from local closure of its instantiated form at depth $n$.

*Substitution.* `Judgement/Subst.lean` establishes that typing is stable under both name substitution and type substitution. The theorem `Typing_substNames` states that, from a derivation $n \vdash P :: \mathcal{G}$, substituting a free name $y$ for a free name $x$ throughout both the process and its hyperenvironment yields the corresponding derivation $n \vdash P\{y//x\} :: \mathcal{G}\{y//x\}$. The required side condition mandates that if there is a pre-existing occurrence of $y$ in the hyperenvironment then $y = x$, in which case the substitution is the identity operation. Consequently, when $x$ and $y$ are distinct, $y$ must be fresh for the hyperenvironment, preventing two distinct linear resources from being assigned the same channel name after substitution.

The theorem `Typing_substTypes` establishes the corresponding result for type substitution. Given a derivation $n + 1 \vdash P :: \mathcal{G}$, a type $A$ that is locally closed at depth $n$, and an admissible substitution index $k \leq n$, substituting $A$ for index $k$ throughout the process and hyperenvironment yields a derivation $n \vdash P\{A//k\} :: \mathcal{G}\{A//k\}$. Thus, type substitution reduces the type-binder level by one while preserving typability. Both substitution proofs proceed by induction on the typing derivation and rely on the corresponding commutation properties of opening and substitution for processes and types.

## 1.2 Semantics

The dynamic behaviour of the $\pi$MLL fragment is formalised in the `Semantics/` namespace. Following the erasure structure described in **??**, the development contains three related transition systems: a typed transition system over typing derivations, and two projected systems describing the induced process and environment behaviour.

- **`TypingStep`$_{\mathtt{m}}$**: the primary, resource-aware LTS over typing derivations (`JudgementStep/Basic.lean`);
- **`ProcStep`$_{\mathtt{m}}$**: the process-level LTS obtained by erasing typing resources (`ProcStep/Basic.lean`);
- **`EnvStep`$_{\mathtt{m}}$**: the resource-level LTS describing the corresponding evolution of hyperenvironments (`EnvStep/Basic.lean`).

This three-level organisation directly reflects the semantic architecture of Montesi and Peressotti [2021]: the paper takes transitions of typing derivations and obtains process-level and environment-level behaviour through erasure. The mechanisation preserves this organisation, representing each level as an inductive relation. The main presentational difference is therefore the formulation of their correspondence. Rather than establishing equality of labelled successor sets under erasure, as in the paper proof, the Lean development formulates the correspondence relationally through projection and lifting lemmas. The complete inductive definitions of the three transition systems are given in **??????**.

*Labels.* Transition labels are formalised in `Labels.lean`, directly realising the label algebra introduced in **??**. Although the operational semantics mechanised here target $\pi$MLL, the action vocabulary is defined for the full $\pi$LL calculus to support future extensions. The free and introduced name functions "f" and "i" are realised as `Lbl.f` and `Lbl.i`, computed by structural recursion on the label and used in the side conditions of the **??????**. The disjointness condition on `par` labels is enforced as the decidable predicate `Lbl.WF`, which appears as an explicit side condition in the parallel synchronisation rules.

The core action type, `Act`, captures observable communication events, including empty communication, channel transmission, type instantiation, and server interaction. Selection and server actions are parameterised by the auxiliary datatype `Mu`. The `Lbl` inductive then extends observable actions with the silent transition $\tau$, explicit session-link labels, and a parallel constructor combining two observable actions directly, reflecting the synchronized action labels used by the parallel rules.

```
inductive Act : Type
  | one       (x : FPName)
  | tensor    (x y : FPName)
  | output    (x : FPName) (A : Types)
  | muBrack   (x : FPName) (μ : Mu)
  | ...

inductive Lbl : Type
```

```
  | tau
  | link  (x y : FPName)
  | act   (p : Act)
  | par   (l l' : Act)
```

The label notation is overloaded through the `HasBracket` and `HasParen` typeclasses. Since labels record concrete observable actions, their subjects are free process names rather than locally nameless channels. The overloading is therefore confined to the label language. Bracketed and parenthesised labels are distinguished by the type of their arguments, allowing empty communication, channel transmission, type communication, and server actions to share a compact notation. The corresponding notation instances are given in **??**.

*1.2.1 The Typed Transition System.* The primary LTS, $\texttt{TypingStep}_{\texttt{m}}$, is a relation between typing derivations. Its constructors mirror the operational rules shown in **??**, but make every transition intrinsically typed by relating a source typing derivation to a target typing derivation.

```
inductive TypingStepₘ :
  Typing n P 𝒢 → Lbl → Typing n' P' 𝒢' → Prop
```

The corresponding process and environment behaviour is captured by the projected relations $\texttt{ProcStep}_{\texttt{m}}$ and $\texttt{EnvStep}_{\texttt{m}}$, which are used in the subsequent session-fidelity development.

As established previously, both environment and hyperenvironment composition are formalised using total list appends. Consequently, the explicit disjointness proofs required by the static `Typing` relation to safely mix environments must propagate directly into the dynamic $\texttt{TypingStep}_{\texttt{m}}$ relation. For instance, the $\texttt{par}_1$ constructor requires explicit hypotheses (`hD1` and `hD2`) to guarantee that the hyperenvironments remain strictly disjoint both before and after the transition:

```
| par₁
    (hD1 : 𝒢.disjoint ℋ) (hD2 : 𝒢'.disjoint ℋ) (disj : l.i ∩ Q.f = ∅)
    (h : TypingStepₘ 𝒟 l 𝒟') :
    TypingStepₘ
      (Typing.mix hD1 𝒟 ℰ) l (Typing.mix hD2 𝒟' ℰ)
```

**Listing 1.** The $\texttt{par}_1$ constructor of the typed transition relation, with explicit pre- and post-transition disjointness premises.

The post-state disjointness proof `hD2` is included explicitly as a proof-engineering convenience. It is recoverable from the pre-state disjointness proof `hD1`, the freshness condition $i(l) \cap Q.f = \emptyset$, the name correspondence invariant `Typing_f_eq_names`, and $\texttt{TypingStep}_{\texttt{m}}\texttt{\_names\_bound}$, which states that the target hyperenvironment contains no names beyond those already present in the source or introduced by the label. Keeping `hD2` as a premise avoids reconstructing this derived disjointness fact repeatedly in later proofs, at the cost of making the transition relation slightly less minimal. The signatures for the lemmas describing transition name bounds are stated in **??**.

Strictly stepping typing derivations can expose structural artifacts that are usually hidden in the paper presentation. For example, in **??**, the intermediate derivation is typed by $\varnothing \mid [y : 1]$ rather than simply $y : 1$, because the **??** rule preserves the empty component contributed by the terminated left process. In Lean, this corresponds to a `Typing.mix` constructor containing a $\texttt{Typing.mix}_{\emptyset}$ sub-derivation. The result is valid, but reducing it to the canonical form requires applying the monoidal identity laws of the hyperenvironment explicitly, rather than by definitional equality. The final $\equiv$ step in the figure, which additionally uses process congruence to identify $\mathbf{0} \mid_p \mathbf{0}$ with $\mathbf{0}$, is not currently automated because structural congruence has not been mechanised. This limitation is discussed further in **??**.

*1.2.2 Projected Transition Systems.* The process and environment transition systems are defined to capture the two erasure views of a typed transition. $\texttt{ProcStep}_{\texttt{m}}$ records the process behaviour obtained by forgetting typing resources, while $\texttt{EnvStep}_{\texttt{m}}$ records the corresponding resource evolution. These relations are later connected to $\texttt{TypingStep}_{\texttt{m}}$ by the simulation theorems in **??**.

*Process steps.* $\texttt{ProcStep}_{\texttt{m}}$ models the syntactic evolution of processes independently of typing resources. In rules involving binders, the transition label exposes the concrete name selected by the action, so the continuation is opened with that name and the required freshness conditions are recorded directly as premises. For example, the `tensor` rule opens the continuation with the name $y$ transmitted by the label, requiring $y$ to be fresh for both the subject channel and the continuation:

```
| tensor (hF : y ∉ {x} ∪ P.f) :
    ProcStepₘ (#x⟦•⟧.P) (x⟦y⟧) P(#y)
```

This differs from the co-finite formulation of the typing rule. The process transition records the concrete name chosen by an operational step, whereas the typing rule provides a derivation for every name outside a finite exclusion set. The typability proof in **??** reconciles these views by instantiating the co-finite premise with the name exposed by the process transition.

*Environment steps.* $\texttt{EnvStep}_{\texttt{m}}$ formalises transitions of typing hyperenvironments, written $\mathcal{G} \xrightarrow{l} \mathcal{H}$. Its prefix rules record the protocol-level effect of observable actions. For example, a free channel typed by 1 is consumed by an empty output action, while a free channel typed by $\bot$ is consumed by an empty input action. The relation is closed under hyperenvironment composition and permutation, reflecting the list-based representation of environments and hyperenvironments.

The synchronisation and restriction constructors capture internal communication between dual channelzs. In particular, once the communicating channel are hidden by restriction, their synchronisation produces a $\tau$-transition on the surrounding hyperenvironment without exposing the internal resource transformation. The subsequent metatheory relates this environment-level behaviour to the intrinsically typed transitions of $\texttt{TypingStep}_{\texttt{m}}$.

## References

Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the $\pi$-calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818